Cloud Computing Course Project

A Serverless Telegram Assistant on Cloudflare — "hermes-mini"

Summary: Build a Telegram bot that records, checks off and deletes daily tasks, understands free-form Persian text through an LLM, and runs entirely on the Cloudflare free plan — with no server of your own to keep alive.

1. Why this project?

Every core concept of the course comes together in one small project:

- Serverless/FaaS — code runs only when an event arrives; no VM, no Docker.
- Database as a service (Cloudflare D1).
- The webhook vs. polling pattern, for connecting to an external service.
- Secret management — variable vs. secret, and where keys must never live.
- Consuming an LLM API and designing a structured-output contract.
- Designing inside free-plan limits — which is an engineering discipline of its own.

2. What is the product?

There is exactly one user: you. You talk to the bot in the messenger and it must understand you like a human would:

- «شیر بخر» ("buy milk") becomes a task.
- «شیر رو خریدم» ("I bought the milk") ticks that task off.
- «کارام چیه» ("what are my tasks?") lists them.
- «همین رو حذف کن» ("delete this one") deletes it.
- «سلام» ("hi") gets a conversational reply.
- There are no commands; understanding the message and deciding the action is entirely the language model's job.
- The bot replies in Persian and state is persistent — across restarts and from any device.

Alternative messenger: the reference integration for this project is Telegram, but hooking up the Iranian messenger **Bale** (a Telegram-style app with its own Bot API following the same pattern — webhook, sending messages, secret token) is also feasible, and has been done before. Finding and reading its documentation is part of your exercise — if you go that way, include your mapping in the report.

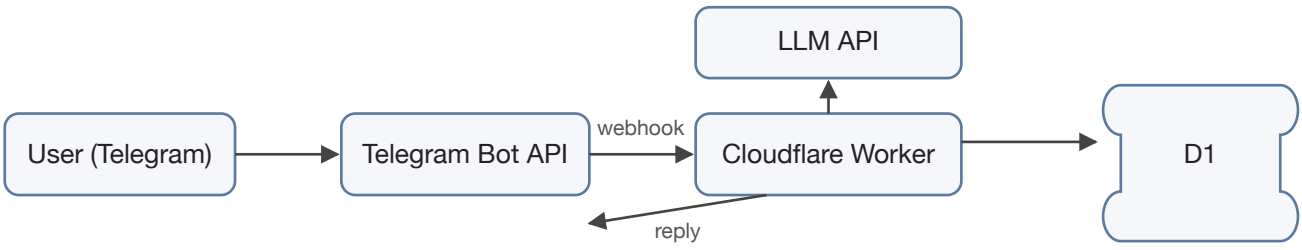
3. Requirements

#	Requirement
R1	All interaction is free-form Persian text — no slash commands at all
R2	Free text → LLM decision → one action from a fixed set (add, done, undone, del, list, chat)
R3	The LLM's output must be structured JSON and must be validated; a malformed output must not crash the bot or write bad data — provide a safe fallback path
R4	Conversational replies (chat) with a minimal conversation memory
R5	Access restricted to a single whitelisted user ID
R6	Webhook protected by a secret — every unsigned request must be rejected
R7	Persistent storage in D1, with migrations kept separate from code
R8	The whole system on the free plan; no payments, no personal servers

4. Hard constraints

- Cloudflare Workers and D1 on the free plan only; no other servers, containers, or paid services.
- Secrets live only in the platform's secret store. If a token has been pushed to the repo, the operations phase scores zero — and you must revoke that token.
- No heavy frameworks; the whole logic fits in a single worker file. If it doesn't fit, you are probably overcomplicating it.

5. Target architecture (high level)



The finer diagram — the boundaries and the decisions inside these boxes — is yours to draw; that is exactly what earns the grade in Phase 1.

6. Phases and grading

Phase	Topic	Points
1	Design and architecture	0.5
2	Worker and secure webhook	0.75
3	D1 database	0.5
4	The intelligence layer (LLM)	1
5	Operations and deployment	0.5
6	Analytical report	0.75
—	**Total**	**4**
+	Bonus (optional)	up to +0.25

Phase 1 — Design and architecture (0.5)

Draw your own architecture diagram and justify these decisions: webhook or polling? Why FaaS for this workload? Why relational data (D1) and not KV? Where does your state live?

Phase 2 — Worker and secure webhook (0.75)

Your worker answers the Telegram webhook; every request without a valid secret header gets a `403`; unauthorized senders are rejected silently; and there is a simple health-check endpoint — on the live URL, not localhost.

Phase 3 — D1 database (0.5)

The table(s) needed for tasks, with a migration kept separate from code. The notion of "today" must follow Tehran time — while your server has no timezone of its own. Document your definition.

Phase 4 — The intelligence layer (1)

Feed free text to an LLM so it returns an action as a single line of JSON; validate the output against your own schema; have a safe path for parse failures. The chat mode must produce conversational replies. In the review we send the bot several challenging messages ("delete all of them", "remove number 3", ambiguous sentences) — robustness is graded exactly here.

Phase 5 — Operations and deployment (0.5)

A clean deploy with wrangler, secrets in the right place, observability turned on, and live logs during the demo. Grading rule: "it works" means a live demo, not a screenshot.

Phase 6 — Analytical report (0.75)

At most 4 pages: the final architecture, a cost analysis at two scales (1 user / 10,000 users), and honestly: where you got stuck and how you unblocked yourself.

Bonus — daily reminder (up to +0.25)

Every morning at a fixed hour, the bot itself sends the day's task list — using Cloudflare's own Cron Triggers and inside the free-plan limits. (Hint: recurring execution and a conditional check of the hour.)

---

7. Common pitfalls — we give the symptom, you find the road

- **The bot is completely silent** and the logs show no incoming request at all ⇒ the problem is upstream of the Worker. One step in the Cloudflare dashboard (Zero Trust related) is a frequent culprit.
- Your own browser cannot open the URL, yet the bot answers messages from Telegram ⇒ that is normal; the workers.dev domain is filtered from Iran (Telegram itself is filtered too — a VPN, which everyone has, solves it). Test with curl/a script, not the browser.
- Deploy fails with error `10405` ⇒ a domain prerequisite has not been created yet.
- Telegram delivers the same message several times ⇒ look at your HTTP response code.

8. Deliverables

1. A git repo with the code and migrations — **with no secrets whatsoever**
2. The live URL of the worker
3. The PDF report (Phase 6)
4. Demo: (in person / a video of at most 3 minutes)

9. References

- Cloudflare Workers — Docs
- Cloudflare D1 and the Wrangler CLI
- Telegram Bot API — especially `setWebhook` and the secret token
- The Iranian messenger **Bale** — a Bot API with the same shape as Telegram's; find its documentation yourself
- Language model: your choice — any LLM with a clean HTTP API (Workers AI, OpenRouter, …)

Using AI tools for learning and debugging is allowed; but in the demo you must defend your architecture and your code.